# PROJECT REPORT:

## DESIGN AND ANALYSIS OF ALGORITHMN


## Member 1:

**Contribution: PART A**

**Name: RAJA SAIF ALI**

**Roll No: i22-1353**

## Member 2:

**Contribution: PART B**

**Name: JUNAID ASRAR**

**Roll No: I22-0770**


## PART A:

## QUESTION 1:

```
Combi(v,c,dp){
        if v<= 2

Return 0

If dp[v][c] is not -1 then

        Return dp[v][c]

S=0

For i:1 to v do

        vp = v - i

        cp = c + i

if  cp > vp then

   s =s+ combi(vp, cp - vp, dp)

else if cp == vp then

   s =s+ combi(cp, 0, dp)

else

   s =s+1 + combi(cp, 0, dp)
```

```
end for
dp[v][c]=s
return s
}
main(){
   read n
   dp = 2D array of size (n+1) x (n+1)
   for i :0 to n do
for j : 0 to n do
        dp[i][j] = -1
end for
   end for
print combi(n, 0, dp)
}
```

**Time Complexity:**

- At each level of recursion, the loop in the combi() function iterates up to "v" times.
- The work done at each level (excluding recursive calls) is O(v).
- The overall time complexity can be written as the sum of work done at each level, which is equal to the sum of the values of "v" encountered during the recursion.
- As "v" decreases with each recursive call, the sum of values of "v" encountered during the recursion can be approximated as O(n^2), where "n"=input size.
- As the size of the input is n. The combi() function is called recursively, and in the worst case, it iterates over "v" values from 1 to "n", where "v" decreases in each recursive call. So, the time complexity of the combi() function is O(n^2).
- Aslo, the 2D array dp is of size (n+1)x(n+1) initialized with -1 using two loop so the time complexity of this is O(n+1)(n+1).
- So, the time complexity of the code is O(n^2).
- Overall time complexity = (n+1)(n+1)+n^2
    - =n^2+n+n+1+n^2
    - =n^2+2n+1+n^2
    - =O(n^2)

**Space Complexity:**

- The space complexity mainly depends on the 2D array dp, which has a size of (n+1) x (n+1), hence the space complexity is O(n^2).

**PART A:**

**QUESTION 2:**

```
val(d[], s, e){
   v = 0
   for i: s to e-1 do
      for j :i+1 to e-1 do
         v = v + d[i] * d[j]
      end for 2
   end for 1
   return v
}


minVal(depo, n, attacks){
   dp = 2D array of size (n+1) x (attacks+1)
   for i: 0 to n do
      dp[i][0] = val(depo, 0, i)
   end for
   for j : 1 to attacks do
      for i : 1 to n do
         dp[i][j] = INT_MAX
         for k : j to i do
            v = val(depo, k, i)
            dp[i][j] = min(dp[i][j], v + dp[k][j - 1])
         end for
      end for
   end for
   return dp[n][attacks]
}
```

```
main(){
   while true do
      read n, attacks
      IF n == 0 and attacks == 0 then
         break
      depo = new array of size n
      for i : 0 to n-1 do
         read depo[i]
      end for
      res = minVal(depo, n, attacks)
      print "Minimum Value: " + res
   end while
}
```

**Time Complexity:**

**Memory Allocation:**

- Allocates memory for a 2D array dp of size (n+1) x (attacks+1).
- The outer loop iterates “n+1” times.
- For each iteration of the outer loop, memory is allocated for size (attacks+1).
- So, the time complexity of memory allocation is O(n * (attacks+1)).

**Initialization:**

- The inner loop iterates “n+1” times.
- For each iteration, a single value is assigned to dp[i][0] by calling the val function.
- The val function has a time complexity proportional to the size of the subarray being processed, which is “I” in this case and ranges to n.
- The sum of squares of integers from 1 to “n” can be represented as (n * (n + 1) * (2n + 1)) / 6.
- So, the time complexity of the initialization is O((n * (n + 1) * (2n + 1)) / 6).

**Calculations:**

Outer Loop:

- Iterates “attacks” times.

Middle Loop:

- Iterates “n” times.

Innermost Loop:

- Iterates from “j” to “I”, where “I” ranges from “1” to “n”.
- For each “I”, “k” ranges from “j” to “I”.
- Within the innermost loop, the val function is called.
- The size of the subarray passed to the val function in each iteration is “i - (k - 1)”.
- Therefore, the time complexity of each call to the val function within the innermost loop is O((i - (k - 1))^2).
- Now, calculate the total complexity of the innermost loop:
- The innermost loop iterates from 'j' to ”I” for each “I”.
- For each iteration of the innermost loop, the val function is called.
- Since “k” ranges from “j” to “I”, the innermost loop has a time complexity proportional to the sum of squares of integers from 1 to “i", which is O(i^3/3).
- Therefore, the overall time complexity of the innermost loop, including the val function called within it, is O(i^3/3).

So, overall time complexity is:

=O(n * (attacks+1) + (n * (n + 1) * (2n + 1)) / 6 + n * attacks * n^3/3)

= O(n^4 * attacks)

**Space Complexity:**

As space complexity involves dp array which is of size (n+1)x(attacks+1). So, the space complexity is O(n*attacks).

**Recursive Formula:**

base case is when “j” is 0, in which dp[i][0]=val(depo,0,i), as no attacks are used.

The recursive formula is given by:

dp[$i$][$j$]=min I to $k$=$j$ (val ($depo$,$k$,$i$) + dp[$k$][$j$−1])

**Order of Solving Sub-Problems:**

I have solved the sub-problems in a topological order, starting from the base cases and gradually moving towards the larger sub-problems.

**Question 3**

```
function findminnum(array, low, high):
 arr = new array of size (high - low + 1)
 for i from 0 to (high - low):
 arr[i] = array[i]
 if arr[0] is not equal to 0:
 return 0
 else:
 for i from low to high:
 if arr[i] is not equal to arr[i + 1] - 1:
 return arr[i] + 1
 return arr[high - 1] + 1
function divide(arr, size):
 left1 = 0, right1 = 0, left2 = 1, right2 = size - 1
 arra = new array of size 4
 found = false
 for i from 0 to size:
 if findminnum(arr, left1, right1) is equal to findminnum(arr, left2, right2):
 found = true
 else:
 increment right1 and left2
 if left2 is equal to right2:
 set arra[0], arra[1], arra[2], arra[3] to -1
 return arra
 if found:
 set arra[0] to left1 + 1, arra[1] to right1 + 1, arra[2] to left2 + 1, arra[3] to right2 + 1
 return arra
function main():
 length = input from user
```

```
arr = new array of size length
for i from 0 to length:
input arr[i] from user
sort arr
finalarray = divide(arr, length)
if finalarray[0] is equal to -1:
print -1
else:
print 2
for i from 0 to 1:
print finalarray[i]
for i from 2 to 3:
print finalarray[i]
```

This program takes an array of integers as input, sorts it, and then divides the array into two parts such that the minimum number in one part is equal to the minimum number in the other part. If such division is possible, it prints "2" followed by the indices of the two parts; otherwise, it prints "-1".

**Time Complexity:**

findminnum function: This function iterates over a subarray of size high - low, which contributes O(n) time complexity.

divide function: This function iterates over the array once, performing constant time operations within the loop, contributing O(n) time complexity.

Overall, the time complexity of the program is dominated by the sorting step, so it is O(n).

**Question 4**

```
function maxPower(M, N, power):
 // Initialize a 2D array for dynamic programming
 create array dp of size M by N
 // Base case: select the first hunter
 for i from 0 to M - 1:
 dp[i][0] = power[i][0]
 // Dynamic programming
 for j from 1 to N - 1:
 for i from 0 to M - 1:
 prevRow = (i - 1 + M) % M
 dp[i][j] = max(dp[i][j - 1], dp[prevRow][j - 1] + power[i][j])
 // Find the maximum total power achieved
 maxPower = INT_MIN
 for i from 0 to M - 1:
 maxPower = max(maxPower, dp[i][N - 1])
 return maxPower
function main():
 // Read input: number of hunters (M) and number of days (N)
 read M, N from input
 // Read power values for each hunter on each day
 create array power of size M by N
 for i from 0 to M - 1:
 for j from 0 to N - 1:
 read power[i][j] from input
 // Call maxPower function and output the result
 result = maxPower(M, N, power)
 output result
```

**Functionality:**

The program defines a function maxPower that takes three arguments: M (the number of hunters), N (the number

of days), and a 2D array power representing the power of each hunter on each day.

It calculates the maximum total power that can be achieved by selecting one hunter each day, such that no two

consecutive days have hunters from the same group.

It uses dynamic programming to solve this problem. The dp array stores the maximum total power achieved up to

the current day for each possible combination of hunters.

The program outputs the maximum total power achieved.

**Analysis:**

The time complexity of this program mainly depends on the dynamic programming part, which involves two nested

loops iterating over all days and all hunters on each day. Therefore, the time complexity is O(M * N), where M is the

number of hunters and N is the number of days.

The space complexity is also O(M * N) because of the dp array.